Project Definition & Development Contract

Building a Vulkan Renderer in C++20

De Montfort University — BSc (Hons) Games Production

Mohamed Deeq Mohamed | P2884884 | 28th March, 2026

Version 9.4 — Vulkan 1.3 Dynamic Rendering; vk-bootstrap throughout; MoSCoW prioritisation; no `VkRenderPass`, no `VkFramebuffer`

1. Student Name:	Mohamed Deeq Mohamed	**2. P-Number:**	P2884884
3. Project Title:	Building a Vulkan Renderer in C++20	**Deadline:**	1 June 2026
4. Supervisor(s):	Salim Hashu / Dr Conor Fahy	**Start:**	30 March 2026

This document serves as both the Project Definition and a formal Development Contract. The objectives stated here form a contract; the final deliverable will be evaluated, in part, against them.

5. Introduction (max. 100 words)

I am a final-year Games Production student at De Montfort University with a strong interest in low-level graphics programming and emerging rendering research. This project involves building a real-time 3D renderer from scratch using the Vulkan 1.3 graphics API and C++20. Vulkan initialisation is handled via **vk-bootstrap**; all rendering code is written directly against the Vulkan API using **Dynamic Rendering** — eliminating `VkRenderPass` and `VkFramebuffer` entirely. The project is both an undergraduate assessment and a deliberate pilot study for my planned PhD in neural rendering at De Montfort University starting October 2026, providing codebase foundations I intend to build on beyond graduation.

6. Aims (max. 100 words)

This project aims to develop a genuine, working understanding of how modern GPUs process and render geometry by implementing a Vulkan 1.3 renderer in C++20. Rather than following a tutorial, I want to make real design decisions and understand their consequences — from synchronisation barriers to descriptor sets to physically-based shading mathematics. Vulkan initialisation boilerplate is abstracted via vk-bootstrap, consistent with professional practice. A secondary aim is to produce a clean, extensible codebase that remains useful after graduation — as a foundation for PhD research in neural rendering, Gaussian splatting, and GPU-accelerated scene reconstruction.

6a. State of the Art (Brief)

Modern real-time graphics is built on physically based rendering (PBR) models that approximate light transport using microfacet BRDFs, such as the Cook–Torrance formulation and later industry-standard refinements like the Disney principled BRDF used in feature animation production. These models link surface roughness, metalness, and view-dependent specular response to physically interpretable parameters, and have become the de facto shading basis for both offline and game engines.

Vulkan 1.3's *dynamic rendering* path, which removes the need for explicit render pass and framebuffer objects, is increasingly recommended as the modern baseline for new engines, simplifying render pass management while retaining low-level control.

In parallel, *neural rendering* has emerged as a major research direction, representing scenes as neural radiance fields (NeRF) that are optimised from multi-view images to enable high-

quality novel view synthesis.[1] While NeRF-style methods achieve impressive visual fidelity, surveys highlight ongoing trade-offs between image quality, training time, and real-time inference performance.

Most recently, *3D Gaussian Splatting* has been proposed as an explicit, GPU-friendly alternative to volumetric NeRFs. Scenes are represented as millions of anisotropic 3D Gaussian primitives, which can be rasterised efficiently to achieve real-time radiance field rendering at 1080p while maintaining state-of-the-art quality.[2] Follow-up work and surveys on Gaussian splatting and neural rendering emphasise the need for robust, extensible rendering back-ends capable of mixing classic rasterisation, PBR shading, and neural scene representations. This project positions a Vulkan 1.3 renderer as such a back-end, with Gaussian splat rendering as an initial bridge towards future PhD research in neural rendering and scene reconstruction.

7. Objectives (max. 200 words)

Each objective is colour-coded by MoSCoW priority (see Appendix B): Must Have Should Have Could Have

1. **Baseline Vulkan Pipeline (Week 2) — *Must Have*:** Set up a Vulkan 1.3 environment via vk-bootstrap and render a coloured triangle using Dynamic Rendering (`vkCmdBeginRendering`), SPIR-V shaders, and manual image layout transitions. No `VkRenderPass` or `VkFramebuffer` anywhere in the project.
2. **Rotating 3D Cube (Week 3) — *Must Have*:** Vertex/index buffers via staging buffer upload, VMA-managed depth image attached dynamically, MVP transformation via UBO and descriptor set.
3. **OBJ Loading and Texture Mapping (Week 5) — *Must Have*:** Load a mesh via `tinyobjloader`, apply a texture through a combined image sampler, use `synchronization2` for layout transitions, and generate a mipmap chain via `vkCmdBlitImage`.
4. **Technical Report and Renderer Polish (Week 9) — *Must Have*:** Interactive camera, swapchain resize handling, zero validation errors, reproducible CMake build, and a complete 8-section technical report including at least one RenderDoc-based performance analysis figure.

5. **PBR Shading (Week 7) — *Could Have*:** Cook–Torrance BRDF (GGX, Smith, Schlick Fresnel) with Dear ImGui runtime controls. Minimum viable: Lambertian diffuse.
6. **Gaussian Splatting (Post Week 7) — *Could Have*:** Render a `.ply` Gaussian splat scene as a PhD research pilot.
7. **XR Exploration (If time permits) — *Could Have*:** Begin OpenXR integration or document it as a direction for further work.

8. Deliverables (max. 100 words)

- A real-time Vulkan 1.3 renderer in C++20 using Dynamic Rendering, featuring OBJ loading, texture mapping with mipmapping, and Dear ImGui runtime controls. PBR shading and Gaussian splat rendering (`.ply` input) pursued as Could Have extensions.
- Existing public GitHub repository created for this project with sustained commit history, reproducible via `CMakePresets.json` on Linux and Windows using vcpkg.
- 8-section technical report covering architecture, pipeline design, PBR mathematics, RenderDoc performance data, and critical reflection.
- Per-milestone evidence packages: annotated screenshots and validation layer logs.

[1] NeRF: Mildenhall et al., "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis", ECCV 2020.

[2] Kerbl et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering", 2023.

9. Schedule of Activities

Week	Focus	Key Tasks
1	Setup & toolchain	CMake + vcpkg (vk-bootstrap), Vulkan SDK, validation layers, RenderDoc, GitHub repo
2	Triangle render	vk-bootstrap init; SPIR-V pipeline; Dynamic Rendering command sequence; layout transition helper
3	Rotating cube	Vertex/index buffers; staging buffer; VMA depth image; UBO + descriptor set; MVP transforms
4	Consolidation + report	RAII audit; begin report §1–2 (Introduction, Architecture)
5	Textured OBJ model	`tinyobjloader`; combined image sampler; `synchronization2`; mipmap generation
6	Report (parallel)	Report §3–4 (Pipeline, Shading Model); BRDF research
7	PBR shading *(Could Have)*	GGX NDF, Smith G, Schlick Fresnel; Dear ImGui panel; Blender reference
7–8	Gaussian splatting *(Could Have)*	`.ply` parsing; GPU splat buffers; depth sort; ellipse projection; alpha compositing
8	Renderer polish	Camera; resize handling; zero validation errors; RenderDoc captures
9	Final submission	Report §5–8; GitHub tidy; CMake verification; supervisor sign-off

10. Requirements

Software: Vulkan SDK 1.3, C++20 (GCC/MSVC/Clang), CMake 3.25+, vcpkg, vk-bootstrap, Vulkan Memory Allocator, tinyobjloader, tinyply, Dear ImGui, GLM, stb_image, spdlog, shaderc/glslc, RenderDoc, JetBrains CLion, Visual Studio 2022, GitHub, Blender (Cycles).

Hardware: GPU with Vulkan 1.3 support (AMD/NVIDIA discrete). Windows 10/11 or Linux (Arch). VR headset access if XR stretch goal is reached (not a hard requirement).

11. Research Hypothesis

> *"Can building a Vulkan rendering pipeline from scratch — using industry-representative abstractions alongside direct API usage — establish the GPU architecture understanding and codebase foundations needed for postgraduate research in neural rendering and AI-driven graphics?"*

I am testing this by designing the renderer to be extensible beyond the assessed milestones: toward Gaussian splatting (a direct precursor to neural rendering), ray tracing, and XR. The technical report's critical reflection section will evaluate whether this approach succeeded.

Supervisory Agreement & Signatures

Party	Signature	Date
12. Student: Mohamed Deeq Mohamed (P2884884)	_____	_____
13. Supervisor: Salim Hashu	_____	_____
Supervisor: Dr Conor Fahy	_____	_____

14. Date: _____

Appendix A — Grade Band Mapping & Milestone Contract

The table below defines the minimum milestone completion required for each grade band. **Could Have milestones (PBR, Gaussian splatting, XR) are not required for any grade band.** Failing to attempt them will not penalise the assessment outcome.

Grade	Mark	Minimum	Rationale
Fail	<40%	Below Obj. 1	No functional Vulkan output
Third	40–49%	Objectives 1–2 + Report §1–3	Working pipeline, triangle, rotating cube
2:2	50–59%	Objectives 1–3 + Report §1–4	Textured OBJ model; full texture pipeline
2:1	60–69%	Objectives 1–3 + Report §1–6 + polish	Camera, resize, RenderDoc analysis, strong report
First	**70%+**	**Objectives 1–4 + full report**	**All 8 report sections; code quality; reflection — PBR and stretch goals not required**

Important: A First Class Honours outcome is achievable through Objectives 1–4 (baseline pipeline through textured OBJ, renderer polish, and the full technical report) without any PBR or stretch goal implementation. Could Have objectives are pursued for personal and research benefit only — they are not assessed differentiators.

#	Milestone	Priority	Grade Band	Target	Sign-off
1	Baseline Vulkan pipeline + triangle	Must Have	Third (40%+)	Week 2	_____
2	Rotating 3D cube	Must Have	2:2 (50%+)	Week 3	_____
3	OBJ loading + texture mapping	Must Have	2:1+ (60%+)	Week 5	_____
4	Renderer polish + full report	Must Have	First (70%+)	Week 9	_____
5	PBR shading	Could Have	Not graded	Week 7	_____
6	Gaussian splatting (`.ply`)	Could Have	Not graded	Post Obj. 3	_____
7	XR/VR exploration	Could Have	Not graded	If time	_____

Appendix B — MoSCoW Prioritisation

Must Have

- Vulkan 1.3 Dynamic Rendering throughout — no `VkRenderPass` or `VkFramebuffer` anywhere.
- Baseline triangle render via `vkCmdBeginRendering` (Objective 1).
- Rotating 3D cube with VMA depth image, UBO, and MVP transforms (Objective 2).

- Textured OBJ model with mipmapping (Objective 3).
- RAII-compliant C++20 codebase with zero validation errors on startup and shutdown.
- Reproducible CMake build on Linux and Windows via vcpkg.
- Public GitHub repository with a sustained commit history.
- Full 8-section technical report (Objective 4).

Should Have

- Interactive orbit or first-person camera.
- Swapchain resize handling (`VK_ERROR_OUT_OF_DATE_KHR`).
- Dear ImGui overlay via second `vkCmdBeginRendering` with `VK_ATTACHMENT_LOAD_OP_LOAD`.
- RenderDoc frame capture analysis with GPU timing data.
- Per-milestone evidence packages (screenshots, validation logs, written notes).

Could Have — Not Required for Any Grade Band

- **PBR shading:** Cook–Torrance BRDF (GGX, Smith G, Schlick Fresnel) with Dear ImGui material controls. Minimum: Lambertian diffuse. Pursued because understanding shading mathematics is directly relevant to my PhD research in neural rendering.
- **Gaussian splat rendering:** Ingest a `.ply` file, sort and project 3D Gaussians to screen-space ellipses, alpha-composite the result. A direct pilot for PhD research in Gaussian-based neural rendering at De Montfort University. Only started once Objectives 1–3 are signed off.
- **VR/AR/XR exploration via OpenXR:** Investigate integrating OpenXR into the renderer. Vulkan is the natural XR backend and Dynamic Rendering is compatible with per-eye render structures. A full implementation is unlikely within the project timeline; even partial progress or a detailed further-work discussion in the report is valuable. Pursued only after Gaussian splatting is complete or formally deferred.

Won't Have — Explicitly Out of Scope

This is a **renderer**, not a game engine. The following will not be implemented: physics simulation, entity-component systems, scene graph, scripting runtime, audio, level editor tooling, networking, shadow mapping, deferred shading, full ray tracing pipeline, mobile/console Vulkan, NeRF training or inference, multi-GPU rendering.

Appendix C — Risk Register

Risk	Likelihood	Impact	Mitigation
Report deprioritised in favour of coding	High	High	Writing begins Week 4; §5 Challenges maintained as a daily log from Day 1 — never written retrospectively.
Vulkan synchronisation errors consuming time	High	Medium	Validation layers always on; `synchronization2` for all barriers; RenderDoc from M1 onwards.
Gaussian splatting scope creep displacing core polish	Medium	High	Obj. 6 strictly not started until Obj. 1–3 signed off. Any time pressure on Obj. 4 immediately suspends it.
Image layout transition errors under Dynamic Rendering	Medium	Medium	Reusable `transitionImageLayout` helper written at Obj. 1 and applied consistently thereafter.
vk-bootstrap incompatibility on DMU lab GPU drivers	Low	Medium	Full fallback control via vk-bootstrap selectors; `uni-debug` preset verified on a lab machine before Obj. 1 sign-off.
CMake/vcpkg build failures across platforms	Low	Medium	Both presets verified on clean environments before submission.

Key: High Medium Low

Appendix D — Student Commitments

- Milestone evidence presented at each weekly check-in.
- Report writing begins no later than **Week 4**; Section 5 (Challenges) maintained as a running log from Day 1.
- vk-bootstrap initialisation sequence documented in Report §2 and §7 to demonstrate understanding regardless of the abstraction used.
- **Objective 6 (Gaussian splatting) not started until Objectives 1–3 are signed off.** Time pressure on Objective 4 immediately pauses all stretch work.
- Scope additions discussed with supervisors before implementation.
- Work tracked in the existing public GitHub repository created for this project, with regular, descriptive commits and use of issues where appropriate.
- Project management run through a dedicated Jira Software project configured with epics mirroring Objectives 1–7 (baseline Vulkan pipeline, rotating cube, textured OBJ model, renderer polish, PBR shading, Gaussian splatting, XR exploration) to keep progress visible and accountable.

Supervisory responsibilities: weekly milestone review (~30 min); sign-off on evidence packages; written report feedback no later than one week before the submission deadline.